

MONOLITHIC SPRING BOOT APPLICATION

Step-by-Step Project Documentation

Conversion of a Microservices-Based Application into a Monolithic Architecture

Project Type	Spring Boot Monolithic Application
Main Modules	User, Product, Order
Testing Tool	Postman
Source Control	Git and GitHub

Table of Contents

1. Project Overview
 2. Project Objective
 3. Architecture
 4. Technologies and Tools
 5. Final Project Structure
 6. Step-by-Step Implementation
 7. User Module
 8. Product Module
 9. Order Module
 10. Running the Application
 11. Postman API Testing
 12. Git and GitHub
 13. Final Verification Checklist
 14. Common Problems and Solutions
 15. Advantages and Limitations
 16. Conclusion
- Appendix – Presentation Explanation

1. Project Overview

This project is a Spring Boot backend application containing User, Product, and Order functionality. The original application used a microservices approach. The task was to implement the same functionality as a monolithic application.

In the final design, all three modules are inside one Spring Boot project and are started as one application. The code is still logically separated into Controller, Service, Repository, and Entity layers.

2. Project Objective

- Understand microservices and monolithic architecture.
- Convert User, Product, and Order functionality into one Spring Boot application.
- Organize code using Controller, Service, Repository, and Entity layers.
- Build and run the application successfully.
- Test APIs using Postman.
- Use Git and GitHub for source-code management.

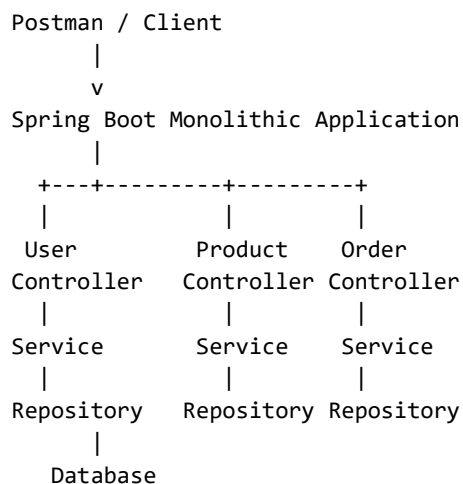
3. Architecture

3.1 Original Microservices Architecture

User, Product, and Order can run as separate services. Each service may have its own process, port, deployment, and database.

3.2 Final Monolithic Architecture

User, Product, and Order are now maintained inside one Spring Boot application. They share the same runtime and application configuration.



4. Technologies and Tools

Technology/Tool	Purpose	Role
Java	Programming language	Backend
Spring Boot	Create/run application	Backend
Spring Web	REST controllers	API
Spring Data JPA	Data access	Persistence

Maven	Dependencies/build	Build
VS Code	Development	Coding
Postman	API testing	Testing
Git	Version control	Source control
GitHub	Remote repository	Hosting

5. Final Project Structure

```

demo/
├── pom.xml
├── src/
│   ├── main/java/com/example/demo/
│   │   ├── DemoApplication.java
│   │   ├── user/
│   │   │   ├── controller/
│   │   │   ├── entity/
│   │   │   ├── repository/
│   │   │   └── service/
│   │   ├── product/
│   │   │   ├── controller/
│   │   │   ├── entity/
│   │   │   ├── repository/
│   │   │   └── service/
│   │   └── order/
│   │       ├── controller/
│   │       ├── entity/
│   │       ├── repository/
│   │       └── service/
│   └── test/
└── docs/

```

6. Step-by-Step Implementation

Step 1 – Open the project

Open the monolithic Spring Boot project in VS Code. Verify pom.xml and src/main/java are present.

Step 2 – Configure Maven

Verify pom.xml contains the required Spring Boot dependencies.

Step 3 – Main application

DemoApplication.java is the entry point and starts the complete application.

Step 4 – User module

Create/maintain User entity, repository, service, and controller.

Step 5 – Product module

Create/maintain Product entity, repository, service, and controller.

Step 6 – Order module

Create/maintain Order entity, repository, service, and controller.

Step 7 – Single application

Keep all three modules under the same Spring Boot project and package hierarchy.

Step 8 – Build

Use Maven to compile/package the project and check for build errors.

Step 9 – Run

Start DemoApplication.java and wait for successful Spring Boot startup.

Step 10 – Test

Use Postman to test User, Product, and Order APIs.

Step 11 – Version control

Stage, commit, synchronize with GitHub if necessary, and push the final code.

7. User Module

The User module manages user information using the layered design:

```
UserController -> UserService -> UserRepository -> User
```

Typical operations tested:

- POST /users – create user
- GET /users – retrieve users
- PUT /users/{id} – update user
- DELETE /users/{id} – delete user

Use the exact mappings and JSON field names from the final source code if they differ.

8. Product Module

The Product module manages product information such as product name, quantity, and price.

```
ProductController -> ProductService -> ProductRepository -> Product
```

- POST /products – create product
- GET /products – retrieve products
- PUT /products/{id} – update product
- DELETE /products/{id} – delete product

9. Order Module

The Order module manages order operations. The final project contains OrderController, OrderService, OrderRepository, and Order entity.

```
OrderController -> /orders  
OrderService -> business logic  
OrderRepository -> data access  
Order -> entity
```

- POST /orders – create order
- GET /orders – retrieve orders

10. Running the Application

10.1 From VS Code

1. Open DemoApplication.java.
2. Click Run above the main method.
3. Wait for Spring Boot startup.
4. Confirm the terminal shows a successful startup message and configured server port.
5. Keep the application running while testing with Postman.

10.2 Using Maven

```
mvn clean package
```

```
java -jar target\<exact-jar-name>.jar
```

Use the exact JAR filename shown inside the target folder.

11. Postman API Testing

Postman is used to verify that requests reach the correct controller and return the expected response.

11.1 User Testing

Example endpoint: <http://localhost:8080/users>

Methods: POST, GET, PUT, DELETE

6. Start the Spring Boot application.
7. Open Postman and select the HTTP method.
8. Enter the endpoint.
9. For POST/PUT, select Body → raw → JSON and provide valid JSON.
10. Click Send and verify the response.

11.2 Product Testing

Example endpoint: <http://localhost:8080/products>

Methods: POST, GET, PUT, DELETE

11. Start the Spring Boot application.
12. Open Postman and select the HTTP method.
13. Enter the endpoint.
14. For POST/PUT, select Body → raw → JSON and provide valid JSON.
15. Click Send and verify the response.

11.3 Order Testing

Example endpoint: <http://localhost:8080/orders>

Methods: POST, GET

16. Start the Spring Boot application.
17. Open Postman and select the HTTP method.
18. Enter the endpoint.
19. For POST/PUT, select Body → raw → JSON and provide valid JSON.
20. Click Send and verify the response.

12. Git and GitHub

The final code was pushed successfully to the GitHub repository. The remote repository was synchronized before the final push.

12.1 Check status

```
git status
```

12.2 Stage changes

```
git add .
```

12.3 Commit

```
git commit -m "Complete monolithic application"
```

12.4 Synchronize when remote is ahead

```
git pull --rebase origin main
```

12.5 Push

```
git push origin main
```

The final terminal result showed a successful push and later 'Everything up-to-date', confirming synchronization.

13. Final Verification Checklist

- ☐ Project opens correctly in VS Code
- ☐ pom.xml is present and dependencies resolve
- ☐ DemoApplication.java starts successfully
- ☐ User API works in Postman
- ☐ Product API works in Postman
- ☐ Order API works in Postman
- ☐ No unresolved Java compilation errors
- ☐ Final changes are committed
- ☐ Latest code is pushed to GitHub
- ☐ Documentation is prepared

14. Common Problems and Solutions

Problem	Solution
Git push rejected – fetch first	Run git pull --rebase origin main, then git push origin main.
'addl' is not a git command	Use git add . with a space before the period.
JAR file cannot be accessed	Check the exact JAR filename in target and use that filename.
404 in Postman	Check URL, HTTP method, @RequestMapping, method mapping, port, and application status.
500 in Postman	Read the Spring Boot terminal stack trace and verify request JSON, service logic, entity and database configuration.
Modified/deleted/untracked files in git status	Review the changes, then use git add . to stage the intended final project.

15. Advantages and Limitations

15.1 Advantages

- Simple deployment – one application to run.
- Easy local development and testing.
- Modules can share configuration and code directly.
- No inter-service network calls are required.
- Suitable for smaller applications and learning projects.

15.2 Limitations

- A change in one module can require redeploying the whole application.
- The application can become large as modules grow.
- Independent scaling is harder than with microservices.
- A major application failure can affect all modules.

16. Conclusion

The project demonstrates the conversion of a microservices-oriented application into a monolithic Spring Boot application. User, Product, and Order functionality are maintained in one application with Controller, Service, Repository, and Entity layers.

The application was started successfully, APIs were tested using Postman, and the final source code was synchronized with GitHub. Therefore, the core development, execution, API testing, and source-control requirements of the task are completed.

Appendix – Short Explanation for Presentation

“This project was initially implemented using microservices architecture. I converted the same functionality into a monolithic Spring Boot application. I combined the User, Product, and Order modules into one application while keeping separate Controller, Service, Repository, and Entity packages. I tested the APIs using Postman, verified successful Spring Boot startup, and pushed the final source code to GitHub using Git.”

Appendix – Main Commands

```
mvn clean package
java -jar target\<exact-jar-name>.jar
git status
git add .
git commit -m "Complete monolithic application"
git pull --rebase origin main
git push origin main
```